Diseño de Bases de Datos

Seguridad e Integridad de datos: Agenda

Transacciones
- Propidades
- Estados

Transacciones monusuarias
- Atomicidad
- Protocolos

Transacciones centralizadas
- Aislamiento
- Consistencia
- Durabilidad

Entornos Concurrentes

Entorno centralizado

- Varias transacciones ejecutándose simultáneamente compartiendo recursos.
- Deben evitarse los mismos problemas de consistencia de datos
- Transacciones correctas, en ambientes concurrente pueden llevar a fallos

Entornos Concurrentes

Seriabilidad

- Garantiza la consistencia de la BD

To Read (a)
a:= a – 50
Write (a)
Read (b)
b := b + 50
Write (b)

T1 Read (a)
temp := a * 0,1
a:= a – temp
Write (a)
Read (b)
b := b + temp
Write (b)

- Resolver To, T1 o T1, To se respeta A+B
- Ahora bien To T1 <> T1 To

Entornos concurrentes

T0 → T 1
BD A = 1000
B = 2000 3000

T0 → transfiere 50

BD A = 950
B = 2050

T1 → transfiere el 10%

BD A = 855
B = 2145 3000

T1 → T 0
BD A = 1000
B = 2000 3000

T1 → transfiere el 10%

BD A = 900
B = 2100

T0 → transfiere 50

BD A = 850
B = 2150 3000

A + B se respeta!

Entornos Concurrentes

Planificación: secuencia de ejecución de transacciones

- Involucra todas las instrucciones de las transacciones
- Conservan el orden de ejecución de las mismas
- Un conjunto de m transacciones generan m! planificaciones en serie
- La ejecución concurrente no necesita una planificación en serie.

Entornos concurrentes

1.	READ(A)	
2.	A := A – 50	
3.	WRITE(A)	
4.		READ(A)
5.		TEMP := A * 0.1
6.		A := A – TEMP
7.		WRITE(A)
8.	READ(B)	
9.	B := B + 50	
10.	WRITE(B)	
11.		READ(B)
12.		B := B + TEMP
13.		WRITE(B)

A + B se conserva

BD A = 1000 3. 950 7. 945
B =2000 10.2050 13. 2145

To 1, A =1000
2 A = 950
8 B = 2000
9 B – 2050

T1 4 A = 950
5 temp = 95
6 A = 945
11 B = 2050
12 B = 2145

1.	READ(A)	
2.	A := A – 50	
3.		READ(A)
4.		TEMP := A * 0.1
5.		A := A – TEMP
6.		WRITE(A)
7.		READ(B)
8.	WRITE(A)	
9.	READ(B)	
10.	B := B + 50	
11.	WRITE(B)	
12.		B := B + TEMP
13.		WRITE(B)

A + B no se conserva

BD A = 1000 6. 900 8.
B =2000 11.2050 13. 2

To 1, A =1000
2 A = 950
9 B = 2000
10 B = 2050

T1 3 A = 1000
4 temp = 100
5 A = 900
7 B = 2000
12 B = 2100

Entornos Concurrentes

Conclusiones

- El programa debe conservar la consistencia
- La inconsistencia temporal puede ser causa de inconsistencia en planificaciones en paralelo
- Una planificación concurrente debe equivaler a una planificación en serie
- Solo las instrucciones READ y WRITE son importantes y deben considerarse.

Entornos Concurrentes

Conflicto en planificaciones serializables

- I1, I2 instrucciones de T1 y T2
  - Si operan sobre datos distintos. NO hay conflicto.
  - Si operan sobre el mismo dato
    - I1 = READ(Q) = I2, no importa el orden de ejecución
    - I1 = READ(Q), I2 = WRITE(Q) depende del orden de ejecución (I1 leerá valores distintos)
    - I1 = WRITE(Q), I2 = READ(Q) depende del orden de ejecución (I2 leerá valores distintos)
    - I1 = WRITE(Q) = I2, depende el estado final de la BD
- I1, I2 está en conflicto si actúan sobre el mismo dato y al menos una es un write. Ejemplos.

Entornos Concurrentes

Definiciones

- **Una Planificación S se transforma en una S´ mediante intercambios de instrucciones no conflictivas, entonces S y S´ son *equivalentes en cuanto a conflictos.***
- Esto significa que si
  - S' es consistente, S también lo será
  - S' es inconsistente, S también será inconsistente
- **S´ es serializable en conflictos si existe S/ son equivalentes en cuanto a conflictos y S es una planificación serie.**

Entornos concurrentes

1.	READ(A)	
2.	A := A – 50	
3.	WRITE(A)	
4.		READ(A)
5.		TEMP := A * 0.1
6.		A := A – TEMP
7.		WRITE(A)
8.	READ(B)	
9.	B := B + 50	
10.	WRITE(B)	
11.		READ(B)
12.		B := B + TEMP
13.		WRITE(B)

A + B se conserva

1.	READ(A)	
2.	A := A – 50	
3.		READ(A)
4.		TEMP := A * 0.1
5.		A := A – TEMP
6.		WRITE(A)
7.		READ(B)
8.	WRITE(A)	
9.	READ(B)	
10.	B := B + 50	
11.	WRITE(B)	
12.		B := B + TEMP
13.		WRITE(B)

A + B no se conserva

Control de Concurrencia

Métodos de control de concurrencia

- Bloqueo
- Basado en hora de entrada

Control de Concurrencia

Bloqueo

- Compartido *Lock_c*(dato)(solo lectura)
- Exclusivo *Lock_e*(dato) (lectura/escritura)
- Las transacciones piden lo que necesitan.
- Los bloqueos pueden ser compatibles y existir simultáneamente (compartidos)

Una transacción debe:

- Obtener el dato (si está libre, o compartido y solicita compartido)
- Esperar (otro caso)
- Usar el dato
- Liberarlo.

T1 a → b

1. Lock_e(a)
3. Read (a)
4. a := a – 50
5. Write (a)
6. Unlock (a)

10. Lock_e(b)
13. Read (b)
14. b := b + 50
15. Write (b)
16. Unlock (b)

T2 a+ b

2. Lock_c(a)
7. Read (a)
8. Unlock (a)
9. Lock_c(b)
11. Read (b)
12. Unlock (b)
17. informar (a+b)

BD A = 1000 1.exc 5. 950 6. libera 6'.comp
8. libera
B = 2000 9.comp 12. libera 12'.exclusivo
15. 2050 16. libera

T1 3. A = 1000
4. A = 950
10. bloquea!
13. B = 2000
14 B = 2050

T2 2. Bloquea!
7. A =950
11. B = 2000

17??????????????????

T1 → T2 o T2 → T1 en serie, no genera problemas

Control de Concurrencia

Si se ejecutan en orden verde, azul, celeste

Que pasa con los resultados

Se deben llevar los bloqueos de las transacciones al comienzo.

Control de Concurrencia

Deadlock

- situación en la que una transacción espera un recurso de otra y viceversa

Control de Concurrencia

Conclusiones:

- Si lo datos se liberan pronto → se evitan posibles deadlock
- Si los datos se mantienen bloqueados → se evita inconsistencia.

Control de Concurrencia

Protocolos de bloqueo

- Dos fases
  - Requiere que las transacciones hagan bloqueos en dos fases:
    - Crecimiento: se obtienen datos
    - Decrecimiento: se liberan los datos
  - Garantiza seriabilidad en conflictos, pero no evita situaciones de deadlock.
  - Como se consideran operaciones
    - Fase crecimiento: se piden bloqueos en orden: compartido, exclusivo
    - Fase decrecimiento: se liberan datos o se pasa de exclusivo a compartido.

Control de Concurrencia

Protocolo basado en hora de entrada

- El orden de ejecución se determina por adelantado, no depende de quien llega primero
- C/transacción recibe una HDE
  - Hora del servidor
  - Un contador
- Si HDE(Ti) < HDE(Tj), Ti es anterior
- C/Dato
  - Hora en que se ejecutó el último WRITE
  - Hora en que se ejecutó el último READ
  - Las operaciones READ y WRITE que pueden entrar en conflicto se ejecutan y eventualmente fallan por HDE.

Algoritmo de ejecución:

- **Ti Solicita READ(Q)**
  - HDE(Ti) < HW(Q): rechazo (solicita un dato que fue escrito por una transacción posterior)
  - HDE(Ti) ≥ HW(Q): ejecuta y se establece HR(Q)=Max{HDE(Ti), HR(Ti)}
- **Ti solicita WRITE(Q)**
  - HDE(Ti) < HR(Q): rechazo (Q fue utilizado por otra transaccion anteriomente y suposu que no cambiaba)
  - HDE(Ti) < HW(Q): rechazo (se intenta escribir un valor viejo, obsoleto)
  - HDE(Ti) > [HW(Q) y HR(Q)]: ejecuta y HW(Q) se establece con HDE(Ti).
- **Si Ti falla, y se rechaza entonces puede recomenzar con una nueva hora de entrada.**

Control de Concurrencia

Casos de Concurrencia. Granularidad

- A registros caso más normal
- Otros casos
  - BD completa
  - Áreas
  - Tablas

Otras operaciones conflictivas

- Delete(Q) requiere un uso completo del registro
- Insert(Q) el dato permanece bloqueado hasta la operación finalice.

Registro Histórico en entornos concurrentes

Consideraciones del protocolo basado en bitácora

- Existe un único buffer de datos compartidos y uno para la bitácora
- C/transacción tiene un área donde lleva sus datos
- El retroceso de una transacción puede llevar al retroceso de otras transacciones

Retroceso en cascada

- Falla una transacción → pueden llevar a abortar otras
- Puede llevar a deshacer gran cantidad de trabajo.

Registro Histórico en entornos concurrentes

Durabilidad

- Puede ocurrir que falle Ti, y que Tj deba retrocederse, pero que Tj ya terminó. Como actuar?
- Protocolo de bloqueo de dos fases: los bloqueos exclusivos deben conservarse hasta que Ti termine.
- HDE, agrega un bit, para escribir el dato, además de lo analizado, revisar el bit si está en 0 proceder, si está en 1 la transacción anterior no termino, esperar….

Registro Histórico en entornos concurrentes

Bitácora

- Similar sistemas monousuarios
- Como proceder con checkpoint
  - Colocarlo cuando ninguna transacción esté activa. Puede que no exista el momento.
  - Checkpoint<L> L lista de transacciones activa al momento del checkpoint.
- Ante un fallo
  - UNDO y REDO según el caso.
  - Debemos buscar antes del Checkpoint solo aquellas transacciones que estén en la lista.